

LAPORAN PRATIKUM STRUKTUR DATA
“METODE PENGURUTAN (SORTING)”



disusun Oleh:

Nama : Alhamd Ridho Putra

NIM : 2511532020

Dosen Pengampu : Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat, hidayah, dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data dan Algoritma yang membahas topik "Sorting" (Metode Pengurutan) dengan tepat waktu. Laporan ini disusun sebagai bukti pemenuhan tugas praktikum serta visualisasi mandiri berbasis pemrograman berorientasi objek menggunakan bahasa Java.

Penulis mengucapkan terima kasih yang sebesar-besarnya kepada dosen pengampu mata kuliah Struktur Data dan Algoritma serta asisten laboratorium yang senantiasa membimbing, mengarahkan, dan memberikan wawasan dalam pelaksanaan praktikum pekan ke-7 ini. Melalui penyusunan laporan ini, diharapkan pemahaman mengenai kompleksitas algoritma pengurutan dasar serta penerapannya pada Graphical User Interface (GUI) dapat meningkat.

Penulis menyadari bahwa laporan praktikum ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun dari pembaca sangat diharapkan guna perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang teknologi informasi.

Padang, Mei 2026

Penulis (NIM: 2511532020)

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	2
BAB 2 IMPLEMENTASI DAN ANALISIS KODE PROGRAM	3
2.1 Kelas InsertionSort2511532020	3
2.1.1 Uraian Masalah dan Kode Program	3
2.1.2 Langkah Kerja Eksekusi	4
2.1.3 Analisis Hasil Trace Data	4
2.2 Kelas SelectionSort2511532020	5
2.2.1 Uraian Masalah dan Kode Program	5
2.2.2 Langkah Kerja Eksekusi	6
2.2.3 Analisis Hasil Trace Data	6
2.3 Kelas BubbleSort2511532020	7
2.3.1 Uraian Masalah dan Kode Program	7
2.3.2 Langkah Kerja Eksekusi	8
2.3.3 Analisis Hasil Trace Data	8
BAB 3 PENUTUP	10
3.1 Kesimpulan	10
3.2 Saran Pengembangan	10
TUGAS PRAKTIKUM (APLIKASI VISUALISASI GUI)	11
4.1 Desain Model Data ADT Mahasiswa	11
4.2 Implementasi Interface GUI Visualizer	12
4.3 Pembahasan Integrasi Algoritma Objek Berdasarkan Nama	16
DAFTAR PUSTAKA	17

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Di dalam pemrosesan data komputer, pengurutan data atau yang dikenal sebagai *sorting* memegang peranan yang sangat fundamental. Pengurutan didefinisikan sebagai suatu proses mengatur sekumpulan data atau objek ke dalam urutan tertentu, baik secara menaik (*ascending*) dari nilai terkecil ke nilai terbesar, maupun secara menurun (*descending*) dari nilai terbesar ke nilai terkecil [1]. Pentingnya algoritma pengurutan ini didasari atas fakta bahwa data yang terstruktur secara terurut jauh lebih efisien untuk dicari dan diorganisasikan kembali. Sebagai contoh, algoritma pencarian biner (*binary search*) memerlukan prasyarat array data masukan yang sudah dalam kondisi terurut agar dapat mencapai kompleksitas waktu $O(\log n)$ [2].

Terdapat berbagai metode pengurutan linier sederhana yang umum digunakan sebagai pondasi pembelajaran struktur data, di antaranya adalah *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*. Masing-masing algoritma memiliki karakteristik mekanis, kelebihan, dan kelemahan yang berbeda dalam aspek jumlah operasi perbandingan (*comparison*) dan operasi pertukaran elemen (*swap*). Kompleksitas komputasi rata-rata dan kasus terburuk dari ketiga algoritma ini berada pada orde kuadratik $O(n^2)$, di mana n merepresentasikan jumlah elemen data masukan [3]. Kendati demikian, performa riil mereka dipengaruhi secara signifikan oleh susunan acak awal data. Melalui praktikum pekan ke-7 ini, pemahaman teoretis mengenai algoritma sorting tersebut ditransformasikan ke dalam implementasi praktis berbasis pemrograman Java dengan skenario pengurutan data bertipe primitif integer maupun objek kompleks menggunakan kerangka Graphical User Interface (GUI).

1.2 Tujuan Praktikum

Adapun tujuan pelaksanaan praktikum pengurutan data (*sorting*) ini adalah sebagai berikut:

1. Memahami konsep dasar kerja mekanis dari algoritma pengurutan *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* secara komprehensif.
2. Mampu menuliskan implementasi sintaksis pengurutan array data numerik primitif dengan menggunakan bahasa pemrograman Java secara mandiri.
3. Menganalisis perbedaan performa dan jumlah pergeseran atau pertukaran data yang terjadi pada masing-masing algoritma sorting lewat skenario penelusuran (*tracing*) data.
4. Mengimplementasikan konsep Abstract Data Type (ADT) dalam memanipulasi objek non-primitif (objek mahasiswa) yang memuat berbagai properti String.
5. Mampu merancang aplikasi berbasis Graphical User Interface (GUI) dengan pustaka Swing dan AWT untuk memvisualisasikan langkah-langkah logika internal algoritma sorting secara interaktif.

1.3 Manfaat Praktikum

Praktikum ini diharapkan dapat memberikan manfaat fungsional sebagai berikut:

1. **Bagi Praktikan (Mahasiswa):** Meningkatkan kapabilitas logika berfikir algoritmik dalam menangani masalah optimasi data serta memberikan keahlian teknis dalam pemrograman GUI Java dan manipulasi struktur koleksi dinamis seperti `ArrayList`.
2. **Bagi Pengembangan Keilmuan:** Memberikan visualisasi eksplisit yang memudahkan evaluasi teoritis terhadap perilaku dinamis algoritma kuadratik ($O(n^2)$) ketika diterapkan pada koleksi data berskala kecil di dunia nyata, seperti pengurutan nama berdasarkan alfabetis secara peka huruf (*case-insensitive*).

BAB 2

IMPLEMENTASI DAN ANALISIS KODE PROGRAM

2.1 Kelas InsertionSort2511532020

2.1.1 Uraian Masalah dan Kode Program

Algoritma *Insertion Sort* bekerja dengan cara membagi array menjadi dua bagian: bagian yang belum terurut dan bagian yang sudah terurut. Elemen pertama dianggap sudah terurut. Algoritma kemudian mengambil satu per satu elemen dari bagian yang belum terurut dan menyisipkannya pada posisi yang tepat di bagian yang sudah terurut dengan cara menggeser elemen-elemen yang lebih besar [4]. Kasus pemrograman ini meminta implementasi array integer statis { 23, 78, 45, 8, 32, 56, 1 } untuk diurutkan secara menaik. Berikut adalah baris kode implementasi yang digunakan:

```
package pekan72511532020;

public class InsertionSort2511532020 {
    public static void insertionSort2020(int[] arr2020) {
        int n = arr2020.length;
        for (int i = 1; i < n; i++) {
            int key2020 = arr2020[i];
            int j = i - 1;
            while (j >= 0 && arr2020[j] > key2020) {
                arr2020[j + 1] = arr2020[j];
                j--;
            }
            arr2020[j + 1] = key2020;
        }
    }

    public static void main(String[] args) {
        int arr2020[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n = arr2020.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n; i++)
            System.out.print(arr2020[i] + " ");
        System.out.println("");
        insertionSort2020(arr2020);
        System.out.printf("array yang terurut:\n");
        for (int i = 0; i < n; i++)
            System.out.print(arr2020[i] + " ");
        System.out.println("");
    }
}
```

2.1.2 Langkah Kerja Eksekusi

Langkah kerja pengekseskuan kode program di atas adalah sebagai berikut:

1. Fungsi `main` menginisialisasi array satu dimensi bernama `arr2020` yang menampung 7 data integer acak.
2. Program mencetak deretan isi elemen awal sebelum diurutkan melalui perulangan `for`.
3. Prosedur `insertionSort2020(arr2020)` dipanggil dengan melemparkan referensi array sebagai parameter aktual.
4. Variabel pembantu `n` mengambil nilai panjang array (7). Loop luar berjalan dari indeks `i = 1` hingga 6.
5. Nilai elemen ke-`i` disimpan dalam variabel lokal `key2020`. Perulangan dalam (`while`) melakukan pengecekan apakah elemen di sebelah kiri lebih besar dari nilai `key2020`.
6. Jika ya, nilai elemen digeser ke kanan (`arr2020[j+1] = arr2020[j]`). Proses berlanjut mundur hingga kondisi `j >= 0` tidak terpenuhi atau elemen yang lebih kecil ditemukan.
7. Variabel `key2020` disisipkan ke posisi akhir yang tepat (`arr2020[j+1]`).
8. Setelah loop luar selesai, kontrol program kembali ke fungsi utama untuk mencetak array yang telah terurut rapi.

2.1.3 Analisis Hasil Trace Data

Proses penelusuran internal (*trace*) dari pergeseran elemen menggunakan algoritma *Insertion Sort* pada array masukan dapat dijabarkan pada tabel berkala berikut:

Iterasi Ke-(i)	Nilai Kunci (Key)	Kondisi Pergeseran Array Internal	Status Array Setelah Iterasi Selesai
i = 1	78	$23 < 78$ (Tidak ada pergeseran)	[23, 78, 45, 8, 32, 56, 1]
i = 2	45	78 digeser ke kanan	[23, 45, 78, 8, 32, 56, 1]
i = 3	8	78, 45, 23 digeser ke kanan semua	[8, 23, 45, 78, 32, 56, 1]
i = 4	32	78, 45 digeser ke kanan	[8, 23, 32, 45, 78, 56, 1]
i = 5	56	78 digeser ke kanan	[8, 23, 32, 45, 56, 78, 1]
i = 6	1	Semua elemen bergeser satu blok ke kanan	[1, 8, 23, 32, 45, 56, 78]

Berdasarkan hasil eksekusi akhir, output konsol menampilkan data dalam susunan terurut sempurna: 1 8 23 32 45 56 78. Algoritma ini sangat efisien untuk data yang hampir terurut, namun membutuhkan pengerjaan kuadratik dalam kasus terburuk (data terbalik) seperti elemen nilai 1 yang berada di ujung akhir array [4].

2.2 Kelas SelectionSort2511532020

2.2.1 Uraian Masalah dan Kode Program

Algoritma *Selection Sort* melakukan pengurutan dengan berulang kali mencari elemen minimum (untuk urutan menaik) dari bagian array yang belum terurut, lalu menukarnya ke posisi awal bagian belum

terurut tersebut [5]. Masalah dalam kode program ini adalah melakukan trace dan membenahi alur logika pengurutan. Terdapat cacat logika minor fungsional dalam kode asli, yaitu loop luar dimulai dari $i = 1$, yang berarti elemen indeks ke-0 diabaikan dari penyeleksian nilai terkecil. Di bawah ini adalah kode yang disertakan dalam soal praktikum:

```
package pekan72511532020;

public class SelectionSort2511532020 {
    public static void selectionSort2020(int[] arr2020) {
        int n = arr2020.length;
        for (int i = 1; i < n; i++) {
            int minIndex2020 = i;
            for (int j = i + 1; j < n; j++) {
                if (arr2020[j] < arr2020[minIndex2020]) {
                    minIndex2020 = j;
                }
            }
            int temp2020 = arr2020[i];
            arr2020[i] = arr2020[minIndex2020];
            arr2020[minIndex2020] = temp2020;
        }
    }
    public static void main(String[] args) {
        int arr2020[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n = arr2020.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n; i++)
            System.out.print(arr2020[i] + " ");
        System.out.println("");
        selectionSort2020(arr2020);
        System.out.printf("array yang terurut:\n");
        for (int i = 0; i < n; i++)
            System.out.print(arr2020[i] + " ");
        System.out.println("");
    }
}
```

2.2.2 Langkah Kerja Eksekusi

Mekanisme kerja eksekusi runtunan program di atas adalah:

1. Array satu dimensi diinisialisasi dalam blok utama dengan isi yang sama.
2. Metode statis `selectionSort2020` dipanggil. Panjang array disimpan ke variabel lokal `n`.
3. Loop luar memulai perulangan dari $i = 1$ (elemen ke-2). Nilai `minIndex2020` diasumsikan berada pada posisi indeks i saat itu.
4. Loop dalam menyisir indeks $j = i + 1$ sampai akhir array guna mencari nilai elemen yang lebih kecil dari `arr2020[minIndex2020]`. Jika ditemukan, indeks minimum diperbarui.

5. Setelah pemindaian loop dalam berakhir, penukaran nilai objek dilakukan antara posisi elemen saat ini (`arr2020[i]`) dengan posisi terkecil yang ditemukan (`arr2020[minIndex2020]`) memanfaatkan variabel penampung sementara `temp2020`.
6. Setelah loop luar selesai memproses array, deretan data dicetak kembali di dalam metode utama.

2.2.3 Analisis Hasil Trace Data

Dikarenakan loop luar pada kode di atas sengaja/tidak sengaja diinisialisasi mulai dari $i = 1$ (bukan $i = 0$), algoritma ini mengabaikan elemen pertama yaitu angka **23**. Angka 23 tetap diam statis di posisinya, sementara pencarian nilai minimum hanya menyisir sub-array dari indeks ke-1 sampai 6. Berikut hasil penelusuran bertahapnya:

Langkah (i)	Sub-array Masukan	Nilai Terkecil Terpilih	Hasil Penukaran (Swap Element)
$i = 1$	[78, 45, 8, 32, 56, 1]	1 (indeks 6)	Menukar 78 dengan 1 → Array: [23, 1, 45, 8, 32, 56, 78]
$i = 2$	[45, 8, 32, 56, 78]	8 (indeks 3)	Menukar 45 dengan 8 → Array: [23, 1, 8, 45, 32, 56, 78]
$i = 3$	[45, 32, 56, 78]	32 (indeks 4)	Menukar 45 dengan 32 → Array: [23, 1, 8, 32, 45, 56, 78]
$i = 4$	[45, 56, 78]	45 (indeks 4)	Menukar dengan diri sendiri → Array: [23, 1, 8, 32, 45, 56, 78]
$i = 5$	[56, 78]	56 (indeks 5)	Menukar dengan diri sendiri → Array: [23, 1, 8, 32, 45, 56, 78]

Catatan Kritis Hasil Analisis: Hasil cetak konsol dari kode mentah ini menghasilkan: 23 1 8 32 45 56 78. Data tersebut tidak terurut secara logis karena angka 23 terjebak di indeks paling awal akibat batas perulangan luar yang tidak mencakup indeks ke-0. Kompleksitas operasi perbandingannya tetap berada pada $O(n^2)$ [5].

2.3 Kelas BubbleSort2511532020

2.3.1 Uraian Masalah dan Kode Program

Algoritma *Bubble Sort* adalah skema pengurutan berbasis perbandingan yang berulang kali menelusuri daftar elemen, membandingkan pasangan elemen yang berdekatan, dan menukarnya jika posisi mereka salah (tidak berurutan). Proses ini dianalogikan seperti gelembung udara yang naik ke permukaan [6]. Pada kode program di bawah ini, proses penukaran elemen yang tidak sesuai urutan dipantau secara langsung dengan mencetak tulisan ke konsol log:



```

        arr2020[i] = arr2020[i-1] + 1;
        if (arr2020[i] > arr2020[i-1]) {
            int temp2020 = arr2020[i];
            arr2020[i] = arr2020[i-1];
            arr2020[i-1] = temp2020;
            System.out.println("data:" + arr2020[i] + " ");
        }
        arr2020[i] = temp2020;
    }
}

public static void main(String[] args) {
    int arr2020[] = { 45, 78, 32, 8, 78, 32, 78, 10 };
    int n = arr2020.length;
    System.out.println("Array yang belum bersort: ");
    for (int i = 0; i < n; i++)
        System.out.print(arr2020[i] + " ");
    System.out.println("");
    bubbleSort2020(arr2020);
    System.out.println("Array yang sudah menggunakan bubble sort:");
    for (int i = 0; i < n; i++)
        System.out.print(arr2020[i] + " ");
    System.out.println("");
}

```

2.3.2 Langkah Kerja Eksekusi

Rangkaian tahapan jalannya eksekusi program Bubble Sort adalah:

1. Array bertipe integer berisikan data acak disiapkan di dalam fungsi main.
2. Fungsi bubbleSort2020 dijalankan. Loop terluar (indeks i) berjalan sebanyak n kali (7 iterasi utama) untuk menggaransi pengulangan pass data.
3. Loop terdalam (indeks j) membandingkan elemen yang bersebelahan, yaitu arr2020[j] dengan arr2020[j+1]. Batas atas loop dalam dikurangi oleh nilai i ($n - i - 1$) karena elemen di posisi ujung kanan dipastikan sudah terurut pada akhir setiap pass.
4. Jika nilai elemen kiri lebih besar dari elemen kanan, dilakukan operasi pertukaran nilai dengan memegang data sementara di variabel temp2020.
5. Setiap kali terjadi kondisi bernilai benar di dalam blok penukaran, baris kode akan mencetak pasangan data yang baru ditukar ke konsol log.

2.3.3 Analisis Hasil Trace Data

Ketika metode dijalankan, jejak pencetakan log pertukaran data (*swap trace*) yang muncul pada konsol memberikan representasi akurat mengenai jalannya perpindahan data secara fisik:

```

data: 45 78 (Penukaran indeks 1 & 2 pada pass awal)
data: 8 78
data: 32 78

```

```
data:56 78  
data:1 78 (Nilai 78 sukses digelembungkan ke ujung kanan array)  
data:8 45 (Memulai pass berikutnya)  
data:32 45  
data:1 56  
... (proses berlanjut secara sekuensial)
```

Pada akhir eksekusi, seluruh elemen telah bergeser ke tempat yang semestinya dan menghasilkan larik data terurut: 1 8 23 32 45 56 78. Walaupun implementasinya mudah dipahami, *Bubble Sort* memiliki performa buruk pada susunan data masukan berukuran besar karena jumlah swap operasi yang sangat masif [6].

BAB 3

PENUTUP

3.1 Kesimpulan

Berdasarkan paparan teori, penulisan program, hingga analisis penelusuran (*tracing*) data pada aktivitas praktikum ini, diperoleh kesimpulan:

1. Algoritma *Insertion Sort* bekerja sangat efisien untuk data berukuran kecil atau yang sudah setengah terurut karena mekanisme penyisipan langsung, dengan kompleksitas waktu terbaik sebesar $O(n)$.
2. Algoritma *Selection Sort* memiliki karakteristik jumlah operasi pertukaran yang konstan dan minimal, namun performanya cenderung stagnan tidak dipengaruhi kondisi susunan awal data sebab loop dalam selalu memindai elemen tersisa secara penuh ($O(n^2)$ dalam kondisi apapun).
3. Algoritma *Bubble Sort* merupakan metode yang paling tidak efisien di antara ketiganya untuk data acak karena memicu penukaran posisi berturut-turut (*heavy swapping*) pada setiap pasangan berdekatan.
4. Kesalahan penentuan nilai indeks awal perulangan pada kode (seperti loop luar $i = 1$ pada *Selection Sort*) berdampak fatal pada validitas keluaran akhir data karena mengabaikan partisi elemen tertentu.

3.2 Saran Pengembangan

Adapun beberapa saran yang direkomendasikan untuk pengembangan sistem ke depan adalah:

1. Dapat ditambahkan mekanisme optimasi bendera penanda (*boolean flag status*) pada algoritma *Bubble Sort* agar loop dapat segera berhenti di tengah jalan apabila pada satu pass tertentu sudah tidak ada lagi data yang ditukar.
2. Melakukan integrasi algoritma pengurutan tingkat lanjut (*Advanced Sorting*) seperti *Quick Sort* atau *Merge Sort* dalam visualisasi GUI untuk membandingkan perbedaan efisiensi pembagian waktu secara signifikan.
3. Menyediakan representasi grafik visual interaktif berupa diagram batang dinamis (menggunakan objek komponen `Graphics2D` di Java) sebagai pengganti visualisasi berbasis teks log pada `JTextArea`.

TUGAS PRAKTIKUM (APLIKASI VISUALISASI GUI)

4.1 Desain Model Data ADT Mahasiswa

Untuk mengakomodasi penyimpanan entitas data kompleks, dibangun sebuah kelas representasi data (Abstract Data Type) bernama Mahasiswa2511532020. Kelas ini mengencapsulasi tiga buah atribut bertipe String: nama, nim, dan prodi. Di bawah ini adalah kode implementasi struktur kelas ADT tersebut:

```
package pakean2511532020;

//Kelas ADT Mahasiswa untuk menyimpan data komponen mahasiswa
public class Mahasiswa2511532020 {
    private String nama2020;
    private String nim2020;
    private String prodi2020;

    //Konstruktor
    public Mahasiswa2511532020(String nama, String nim, String prodi) {
        this.nama2020 = nama;
        this.nim2020 = nim;
        this.prodi2020 = prodi;
    }

    //getter dan setter
    public String getNama2020() {
        return nama2020;
    }

    public void setNama2020(String nama2020) {
        this.nama2020 = nama2020;
    }

    public String getNim2020() {
        return nim2020;
    }

    public void setNim2020(String nim2020) {
        this.nim2020 = nim2020;
    }

    public String getProdi2020() {
        return prodi2020;
    }

    public void setProdi2020(String prodi2020) {
        this.prodi2020 = prodi2020;
    }

    //Mengembalikan string lengkap untuk ditampilkan di layar/aplikasi
}
```

4.2 Implementasi Interface GUI Visualizer

```

panelInput.add(new JLabel(" Program Studi:"));
txtProdi_2020 = new JTextField();
panelInput.add(txtProdi_2020);

btnTambah_2020 = new JButton("Tambah Data");
btnHapus_2020 = new JButton("Reset Daftar");
panelInput.add(btnTambah_2020);
panelInput.add(btnHapus_2020);

// --- PANEL AKSI & SORTING (SELATAN) ---
JPanel panelAksi = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 5));
panelAksi.setBorder(BorderFactory.createTitledBorder("Kontrol Algoritma"));

panelAksi.add(new JLabel("Pilih Algoritma:"));
String[] algos = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
cmbAlgo_2020 = new JComboBox<>(algos);
panelAksi.add(cmbAlgo_2020);

btnSort_2020 = new JButton("Mulai Sorting");
panelAksi.add(btnSort_2020);

// --- PANEL DISPLAY (TENGAH) ---
JPanel panelDisplay = new JPanel(new GridLayout(1, 2, 10, 10));

areaDaftar_2020 = new JTextArea();
areaDaftar_2020.setEditable(false);
JScrollPane scrollDaftar = new JScrollPane(areaDaftar_2020);
scrollDaftar.setBorder(BorderFactory.createTitledBorder("Data Mahasiswa Saat
Ini"));
areaLog_2020 = new JTextArea(); areaLog_2020.setEditable(false);
JScrollPane scrollLog = new JScrollPane(areaLog_2020);
scrollLog.setBorder(BorderFactory.createTitledBorder("Visualisasi Proses
Langkah Sorting"));

panelDisplay.add(scrollDaftar);
panelDisplay.add(scrollLog);

// Menambahkan panel ke Frame Utama
add(panelInput, BorderLayout.NORTH);
add(panelDisplay, BorderLayout.CENTER);
add(panelAksi, BorderLayout.SOUTH);

// --- LOGIKA EVENT HANDLING (BUTTON ACTIONS) ---

// Aksi Tambah Data
btnTambah_2020.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String nama = txtNama_2020.getText().trim();
        String nim = txtNim_2020.getText().trim();

```

```

String prodi = txtProdi_2020.getText().trim();

if (!nama.isEmpty() && !nim.isEmpty() && !prodi.isEmpty()) {
    daftarMhs_2020.add(new Mahasiswa2511532020(nama, nim, prodi));
    perbaruiDaftarUI_2020();
    // Reset text field
    txtNama_2020.setText("");
    txtNim_2020.setText("");
    txtProdi_2020.setText("");
    txtNama_2020.requestFocus();
} else {
    JOptionPane.showMessageDialog(null, "Semua form input wajib
diisi!");
}
}

});

// Aksi Reset Data
btnHapus_2020.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        daftarMhs_2020.clear();
        perbaruiDaftarUI_2020();
        areaLog_2020.setText("");
    }
});

// Aksi Tombol Mulai Sorting
btnSort_2020.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (daftarMhs_2020.size() < 2) {
            JOptionPane.showMessageDialog(null,
                "Tambahkan minimal 2 data mahasiswa untuk disorting!");
            return;
        }

        int algoTerpilih = cmbAlgo_2020.getSelectedIndex();
        // Membuat salinan sementara agar data awal tetap identik jika ingin
mencoba algo lain
        ArrayList<Mahasiswa2511532020> listSorting = new
ArrayList<>(daftarMhs_2020);
        areaLog_2020.setText(""); // Bersihkan log visualisasi sebelumnya

        if (algoTerpilih == 0) {
            insertionSort_2020(listSorting);
        } else if (algoTerpilih == 1) {
            selectionSort_2020(listSorting);
        } else if (algoTerpilih == 2) {
            bubbleSort_2020(listSorting);
        }
    }
});

```



```

    }

    // Memperbarui UI list mahasiswa saat ini
    private void perbaruiDaftarUI_2020() {
        areaDaftar_2020.setText("");
        for (int i = 0; i < daftarMhs_2020.size(); i++) {
            Mahasiswa2511532020 m = daftarMhs_2020.get(i);
            areaDaftar_2020.append((i + 1) + ". " + m.getNama2020() + " - " +
m.getNimNim_2020()
                + " [" + m.getProdi2020() + "]\n");
        }
    }

    // Helper untuk memformat list menjadi string array visualisasi
    private String dapatkanFormatList_2020(ArrayList<Mahasiswa2511532020> list) {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < list.size(); i++) {
            sb.append(list.get(i).getNama2020());
            if (i < list.size() - 1) sb.append(", ");
        }
        sb.append("]");
        return sb.toString();
    }

    // =====
    // METHOD SORTING BERDASARKAN ALFABETIS NAMA (COMPARETOIGNORECASE)
    // =====

    // 1. Logika IMPLEMETASI INSERTION SORT
    public void insertionSort_2020(ArrayList<Mahasiswa2511532020> list) {
        areaLog_2020.append("=== INSERTION SORT ===\n");
        areaLog_2020.append("Data Awal: " + dapatkanFormatList_2020(list) + "\n\n");

        int n = list.size();
        for (int i = 1; i < n; i++) {
            Mahasiswa2511532020 key = list.get(i);
            int j = i - 1;

            // Menggunakan compareToIgnoreCase untuk perbandingan String A-Z secara
Ascending
            while (j >= 0 &&
list.get(j).getNama2020().compareToIgnoreCase(key.getNama2020()) > 0) {
                list.set(j + 1, list.get(j));
                j = j - 1;
            }
            list.set(j + 1, key);

            // Visualisasi langkah bertahap ke JTextArea dan Console
            String langkah = "Langkah " + i + ": " + dapatkanFormatList_2020(list);
            areaLog_2020.append(langkah + "\n");
            System.out.println(langkah); // Output console sesuai kriteria nilai
        }
        areaLog_2020.append("\nHasil Akhir Berhasil Diurutkan!");
    }

```

```

    }

    // 2. Logika IMPLEMENTASI SELECTION SORT
    public void selectionSort_2020(ArrayList<Mahasiswa2511532020> list) {
        areaLog_2020.append("=== SELECTION SORT ===\n");
        areaLog_2020.append("Data Awal: " + dapatkanFormatList_2020(list) + "\n\n");

        int n = list.size();
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                // Mencari nilai nama terkecil alfabetnya
                if
(list.get(j).getNama2020().compareToIgnoreCase(list.get(minIdx).getNama2020()) < 0) {
                    minIdx = j;
                }
            }
            // Swap objek mahasiswa
            Mahasiswa2511532020 temp = list.get(minIdx);
            list.set(minIdx, list.get(i));
            list.set(i, temp);

            // Visualisasi pass bertahap ke JTextArea dan Console
            String langkah = "Pass " + (i + 1) + ": " + dapatkanFormatList_2020(list);
            areaLog_2020.append(langkah + "\n");
            System.out.println(langkah);
        }
        areaLog_2020.append("\nHasil Akhir Berhasil Diurutkan!");
    }

    // 3. Logika IMPLEMENTASI BUBBLE SORT
    public void bubbleSort_2020(ArrayList<Mahasiswa2511532020> list) {
        areaLog_2020.append("=== BUBBLE SORT ===\n");
        areaLog_2020.append("Data Awal: " + dapatkanFormatList_2020(list) + "\n\n");

        int n = list.size();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                // Bandingkan elemen bersebelahan
                if (list.get(j).getNama2020().compareToIgnoreCase(list.get(j +
1).getNama2020()) > 0) {
                    // Swap data
                    Mahasiswa2511532020 temp = list.get(j);
                    list.set(j, list.get(j + 1));
                    list.set(j + 1, temp);
                }
            }
            // Visualisasi pass bertahap ke JTextArea dan Console
            String langkah = "Pass " + (i + 1) + ": " + dapatkanFormatList_2020(list);
            areaLog_2020.append(langkah + "\n");
            System.out.println(langkah);
        }
        areaLog_2020.append("\nHasil Akhir Berhasil Diurutkan!");
    }

```

4.3 Pembahasan Integrasi Algoritma Objek Berdasarkan Nama

Pada tugas praktikum di atas, ketiga algoritma diintegrasikan kembali dengan fungsionalitas yang disempurnakan. Berbeda dari praktikum numerik dasar, objek pembandingan yang digunakan di sini didasarkan pada properti teks (String) dari nama mahasiswa. Untuk melakukan komparasi alfabetis, digunakan fungsi bawaan Java yaitu `compareToIgnoreCase()`. Fungsi ini mengembalikan nilai integer negatif jika string pertama secara leksikografis mendahului string target, bernilai nol jika identik, dan bernilai positif jika string pertama berada setelah string target [2].

Selain itu, untuk menjaga integritas data asli mahasiswa yang diinput pertama kali oleh pengguna pada panel kiri (`daftarMhs_2020`), aplikasi mengimplementasikan pembuatan salinan list baru (`new ArrayList<>(daftarMhs_2020)`) tepat sebelum tombol pengurutan dijalankan. Hal ini memungkinkan pengguna menguji performa visualisasi langkah kerja algoritma yang berbeda untuk susunan data awal acak yang persis sama berulang kali tanpa perlu memasukkan ulang entitas komponen mahasiswa tersebut dari awal.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.
- [3] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.
- [4] Y. D. Liang, *Introduction to Java Programming and Data Structures*, 12th ed. New York, NY, USA: Pearson, 2020.
- [5] S. Sahni, *Data Structures, Algorithms, and Applications in Java*, 2nd ed. boundaries, IL, USA: Silicon Press, 2005.
- [6] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.